

AI Agents & Agentic AI

LLMs • RAG • Tool Use • Memory • Planning • Multi-Agent Systems • Production Reliability • Interview Prep

A practical handbook combining the supplied visual references with deeper explanations, system design guidance, step-by-step build patterns, detailed examples, failure modes, production considerations, and interview questions.

What this handbook covers

- How LLMs work at a practical engineering level.
- What RAG adds, when to use it, and where it fails.
- What makes an AI agent different from a normal LLM workflow.
- What agentic AI means and how it differs from a single agent.
- How to build agents and agentic systems step-by-step.
- Three detailed AI-agent examples and three detailed agentic-AI examples.
- Memory, tools, planning, orchestration, guardrails, observability, evaluation, and security.
- Common production failures and how to detect and fix them.
- Interview questions with strong answer points.

Part I - Original Reference Sheets

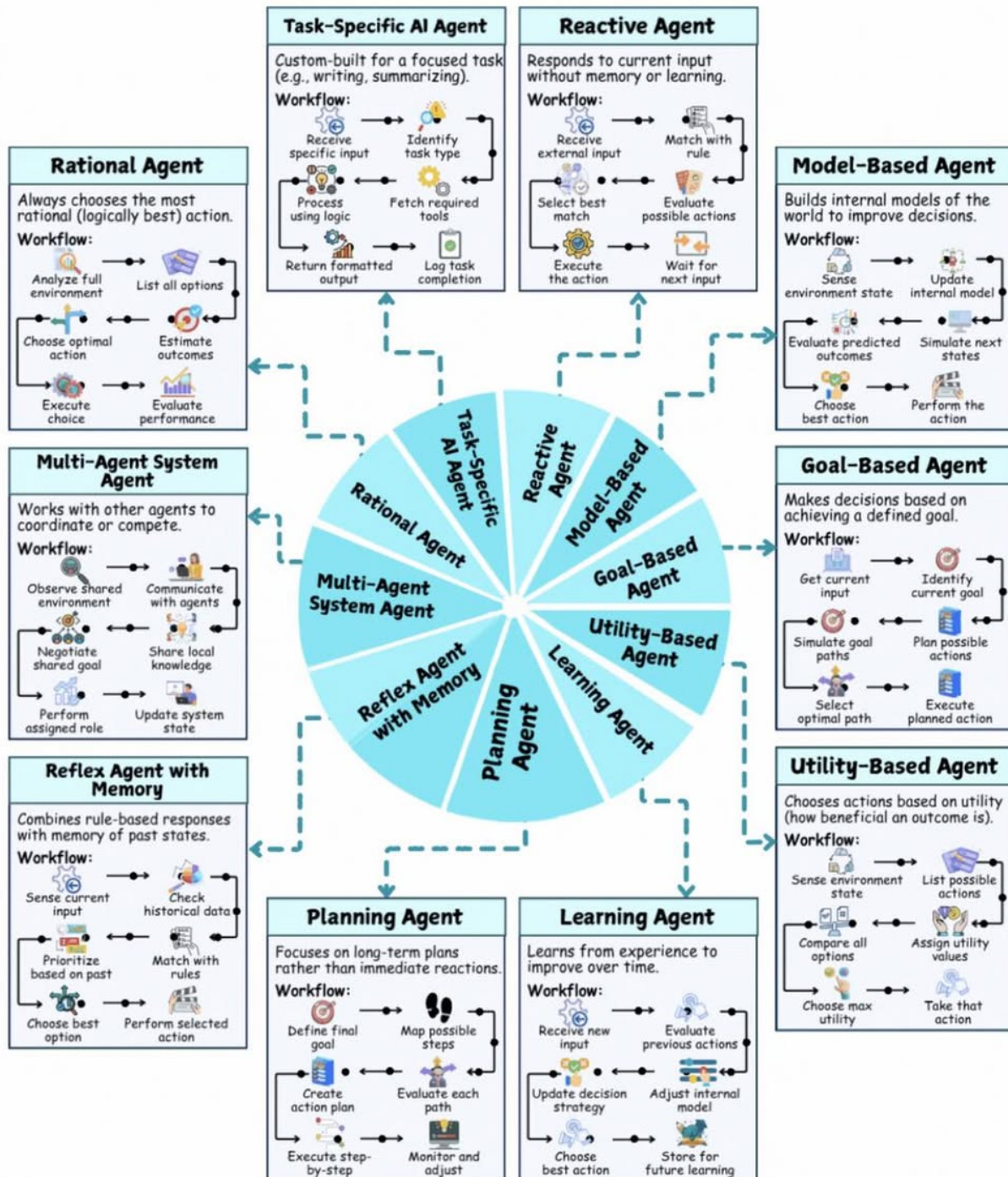
The following pages preserve the five visual references you supplied.

Top 10 Types of AI Agents

Top 10 Types of AI Agents



Shalini Goyal
@goyalshalini

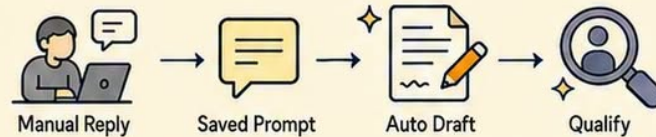


Business AI Agent Examples

The Evolution of AI Agents for Business

6 client and customer agents businesses can use now

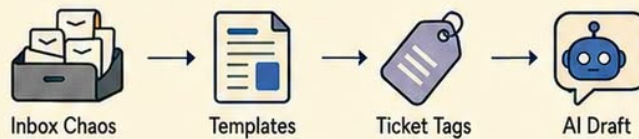
1 Lead Response Agent



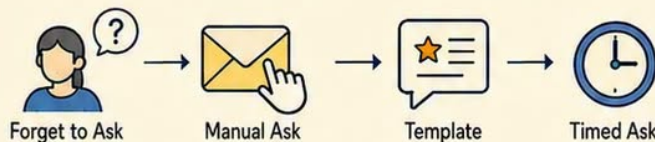
2 Booking Agent



3 Support Agent



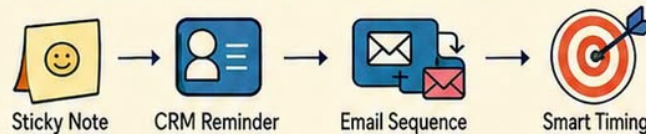
4 Review Agent



5 Onboarding Agent



6 Follow-Up Agent



Comment **AGENTS**



AI MATT



AI AGENT TEAM AUTOMATIONS

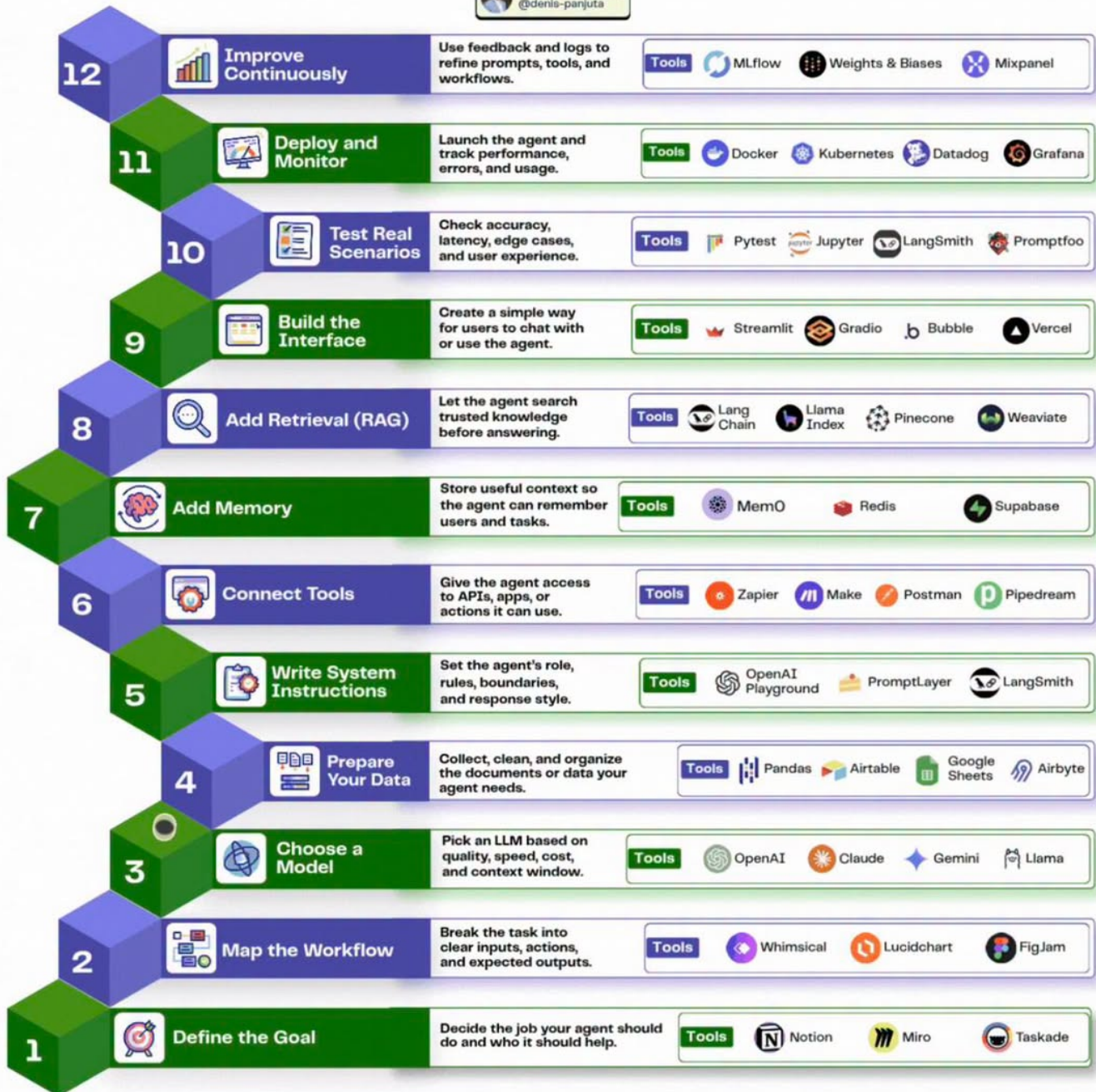


NEXT STEP AGENTS

12 Practical Steps to Build an AI Agent

12 Practical Steps to Create Your First AI Agent

Ideas → Build → Test → Deploy → Improve



Agentic AI vs AI Agents

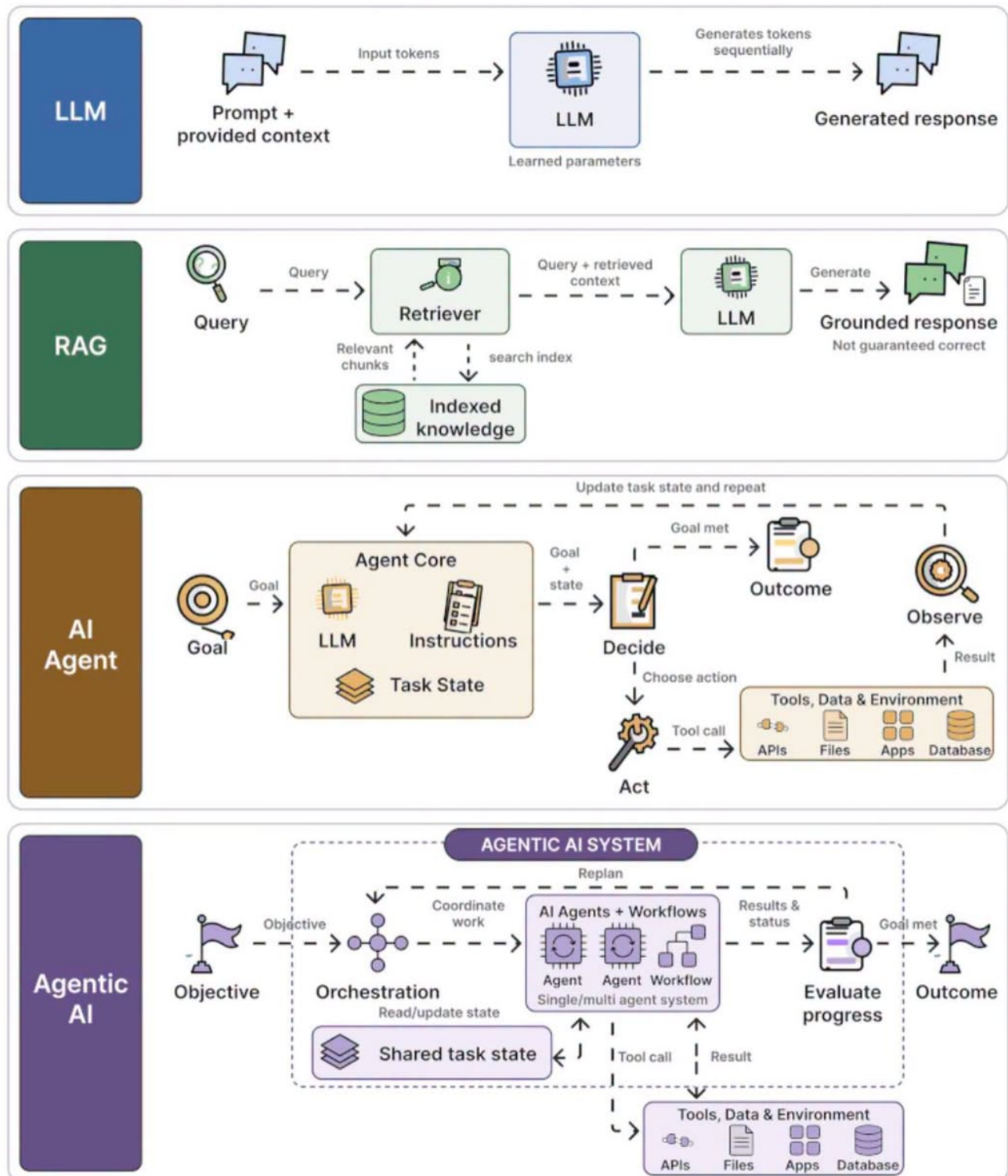
Aspect	Agentic AI	AI Agent
Goal-Orientation	Goal-driven, solves problems on its own	Task-specific, follows set instructions
Decision-Making Process	Makes decisions based on reasoning and analysis	Pre-programmed responses to inputs
Autonomy Level	Highly autonomous, can act independently	Limited autonomy, needs human input
Complexity	Handles complex, dynamic environments	Handles simpler, more structured tasks
Learning Capabilities	Continuously learns and improves	May not learn or only learns within set rules
Responsiveness to Change	Changes its goals and methods autonomously	Limited ability to adapt to new situations
Interaction with Environment	Actively adapts to surroundings and changes	Reacts to set inputs but doesn't adapt



LLM, RAG, AI Agent & Agentic AI

LLM, RAG, AI Agent & Agentic AI

ByteByteGo



Part II - The Core Mental Model

A useful way to think about modern AI systems is as a progression from a model that generates text to a system that can retrieve information, call tools, maintain state, plan, act, observe results, and repeat.

Layer	What it does	Typical limitation
LLM	Generates tokens from prompt + context.	No guaranteed access to fresh or private facts; can hallucinate.
RAG	Retrieves relevant knowledge and adds it to the prompt.	Quality depends on retrieval, chunking, indexing, and source freshness.
AI Agent	Uses an LLM plus tools/state to decide and execute actions toward a goal.	Can loop, choose poor actions, or misuse tools without guardrails.
Agentic AI System	Coordinates one or more agents/workflows with planning, state, evaluation, and re-planning.	Greater complexity, cost, latency, observability, and governance burden.

Simple progression: Prompt → Retrieve → Reason → Decide → Act → Observe → Update state → Repeat.

Part III - LLM Fundamentals

An LLM is a probabilistic sequence model. Given tokens representing the prompt and context, it predicts a distribution over the next token, samples or selects one, appends it, and repeats.

What matters in practice

- **Context window:** how much prompt, retrieved content, tool output, and conversation history can fit in one inference.
- **Tokenization:** text is split into tokens; cost and latency are often token-driven.
- **Temperature / sampling:** affects randomness. Lower values are usually better for deterministic operational workflows.
- **System instructions:** define role, constraints, tool behavior, output format, and safety boundaries.
- **Structured output:** JSON schema or typed output reduces downstream parsing failures.
- **Reasoning quality:** depends on the model, prompt, available context, tool design, and task complexity.

Typical LLM failure modes

Failure	What it looks like	Mitigation
Hallucination	Confident but unsupported answer.	Ground with RAG/tools; require citations; add verification step.
Context overflow	Earlier instructions or evidence disappear.	Summarize state; retrieve on demand; keep structured memory.
Prompt injection	Retrieved/user text attempts to override instructions.	Separate trusted/untrusted data; tool allowlists; policy checks.
Invalid output	Malformed JSON or schema mismatch.	Structured output, validation, retry with repair prompt.
Non-determinism	Same prompt yields different answers.	Lower temperature; constrain output; deterministic post-processing.

Part IV - Retrieval-Augmented Generation (RAG)

RAG combines retrieval with generation. Instead of asking the model to answer only from its training, the system searches a knowledge source and inserts relevant passages into the model context.

Canonical RAG flow

```
User query
-> query transformation (optional)
-> embedding / keyword search
-> retrieve top-k chunks
-> rerank / filter
-> build grounded prompt
-> LLM answer
-> citations / source links
```

Key design choices

- **Chunking:** split documents by semantic boundaries rather than arbitrary lengths when possible.
- **Embeddings:** represent text in vector space so semantically similar content can be retrieved.
- **Hybrid search:** combine vector similarity with keyword/BM25 for better recall.
- **Metadata filters:** limit by tenant, product, date, permissions, region, or document type.
- **Reranking:** score retrieved chunks again using a stronger relevance model.
- **Citations:** preserve source identifiers so answers can be audited.
- **Freshness:** update or re-index when source data changes.

RAG failure modes

Failure	Detection	Fix
Retriever misses answer	Relevant document exists but is never returned.	Improve chunking, query rewrite, embeddings, hybrid search, top-k.
Wrong chunk ranks first	Answer cites irrelevant context.	Add reranker, metadata filters, better scoring.
Stale index	Answer uses old policy/data.	Incremental indexing, freshness timestamps, source-of-truth checks.
Context stuffing	Too many chunks reduce answer quality.	Select fewer high-signal chunks; summarize; deduplicate.
Permission leakage	User retrieves data they should not see.	Enforce authorization before retrieval, not only after generation.

Part V - What Is an AI Agent?

An AI agent is a goal-directed software system that combines an LLM with instructions, task state, tools, and an execution loop. The critical difference from a chatbot is that the system can choose actions and change the external world through tools.

Core loop

```
Goal
-> Observe current state
-> Decide next action
-> Call tool / API
-> Observe result
-> Update task state
-> Check if goal is satisfied
-> Repeat or stop
```

Core components

- **Model:** reasoning and language component.
- **System instructions:** role, rules, boundaries, stop conditions.
- **Tools:** APIs, databases, file operations, search, messaging, ticketing, code execution.
- **State:** current goal, completed steps, intermediate results, errors.
- **Memory:** durable facts or prior interaction summaries that may be useful later.
- **Planner:** optionally decomposes large goals into subtasks.
- **Evaluator:** checks whether an action/output meets quality or safety criteria.
- **Guardrails:** validation, access control, allowlists, human approval, rate/cost limits.
- **Observability:** traces model calls, tool calls, latency, token usage, retries, and outcomes.

Part VI - Common AI Agent Types

Task-specific agent: Optimized for a narrow workflow such as summarization, triage, or document drafting.

Reactive agent: Responds to current input with little or no durable state.

Model-based agent: Maintains an internal representation of the environment and predicts consequences.

Goal-based agent: Selects actions that move toward an explicit target state.

Utility-based agent: Chooses among valid actions using a utility/cost function.

Learning agent: Updates behavior from feedback or historical outcomes.

Planning agent: Creates and revises multi-step plans before or during execution.

Reflex agent with memory: Uses current input plus recent/historical state.

Multi-agent system: Multiple specialized agents coordinate or hand work to one another.

Rational agent: Selects the action expected to produce the best result under defined criteria.

These categories overlap. A single production agent can be goal-based, planning-oriented, memory-enabled, and utility-aware at the same time.

Part VII - How to Build an AI Agent

A production-quality agent should be designed as an engineering system, not just a prompt.

- 1. Define the goal** - Write a measurable outcome and explicit non-goals.
- 2. Define inputs and outputs** - Specify schemas, required fields, and success criteria.
- 3. Map the workflow** - List deterministic steps, decision points, external systems, and failure branches.
- 4. Choose the model** - Balance reasoning quality, latency, context size, privacy, and cost.
- 5. Write system instructions** - Set role, tool rules, forbidden actions, and stopping conditions.
- 6. Add tools** - Expose narrow, typed actions rather than one unrestricted shell/API.
- 7. Add memory only if needed** - Separate short-term task state from long-term user/system memory.
- 8. Add RAG when answers need external knowledge** - Authorize before retrieving; preserve citations.
- 9. Add validation and guardrails** - Schema checks, allowlists, limits, confirmation for risky actions.
- 10. Test realistic scenarios** - Happy path, edge cases, malformed inputs, permission errors, dependency outages.
- 11. Deploy and observe** - Trace every model/tool step; track outcome metrics, latency, retries, token cost.
- 12. Improve continuously** - Use failed traces and user feedback to refine prompts, tools, retrieval, and policies.

Part VIII - Detailed AI Agent Examples

Example 1 - Support Ticket Triage Agent

Goal: classify incoming support tickets, identify severity, retrieve relevant runbooks, draft a response, and route to the correct queue.

Inputs: Ticket text, customer/account metadata, service name, recent incident context.

Tools: Ticket API, knowledge-base search, service catalog, incident-status API.

RAG: Retrieve current runbooks and known-issue articles using service + error signature.

Decision logic: Classify category and severity; if confidence is low or P1/P2, escalate to human.

Actions: Add labels, assign queue, attach suggested runbook, draft response.

Guardrails: Never close tickets automatically; do not expose internal-only content to customers.

Metrics: Routing accuracy, first-response time, escalation precision, human edit rate.

Operational failure: Failure example: the retriever returns an outdated runbook. Detect via stale source timestamp or agent trace. Fix by enforcing freshness metadata and preferring active documents.

Example 2 - SRE Incident Investigation Agent

Goal: assist responders by collecting evidence and proposing the most likely failing layer without making unsafe production changes.

Inputs: Alert, service, environment, cluster/account, time window.

Tools: CloudWatch, Kubernetes read-only commands, logs, metrics, deployment history, DNS/TCP probes.

Workflow: Gather symptoms -> recent changes -> dependency health -> logs/metrics -> ranked hypotheses.

Actions: Generate incident timeline, suggest next checks, produce a manager update.

Guardrails: Read-only by default; restarts/rollbacks require explicit human approval.

Metrics: Time to first useful hypothesis, MTTR reduction, false-hypothesis rate, tool-call success rate.

Operational failure: Failure example: the agent over-focuses on CPU and misses an IAM regression. Fix by using a deterministic diagnostic checklist before free-form reasoning.

Example 3 - Employee Onboarding Agent

Goal: orchestrate a new employee onboarding checklist across HR, IAM, calendar, and team documentation.

Inputs: Employee identity, start date, role, manager, region.

Tools: HR system, IAM request API, calendar, email, document platform.

Workflow: Validate record -> determine required access -> create tasks -> schedule orientation -> track completion.

Actions: Draft emails, create approved access requests, open checklist items.

Guardrails: Privileged access always needs manager/security approval; no role escalation beyond policy.

Metrics: Onboarding completion time, missed-task rate, manual touches, approval turnaround.

Operational failure: Failure example: an API times out after creating an access request but before returning success. The agent must use idempotency keys/status lookup before retrying to avoid duplicate requests.

Part IX - What Is Agentic AI?

Agentic AI is a broader system pattern in which one or more agents autonomously coordinate multi-step work toward an objective. It usually includes orchestration, shared state, evaluation, re-planning, and specialized roles.

Single agent vs agentic system

Aspect	Single AI Agent	Agentic AI System
Scope	One bounded task/workflow	Broader objective with multiple workstreams
Coordination	Mostly internal loop	Orchestrator + multiple agents/workflows
State	Local task state	Shared state across agents and steps
Planning	Optional	Usually explicit and iterative
Evaluation	Often one final check	Continuous evaluation and re-planning
Complexity	Moderate	High
Best fit	Known workflows	Dynamic, multi-domain objectives

Important nuance: agentic AI does not have to mean a swarm of agents. A single agent with robust planning, state, evaluation, and iterative execution can be agentic.

Part X - How to Build Agentic AI

- 1. Define the objective and boundaries** - State success, constraints, budget, allowed systems, and human approval points.
- 2. Decompose responsibility** - Identify specialized roles only where separation improves reliability or expertise.
- 3. Choose orchestration pattern** - Supervisor, planner-executor, sequential pipeline, parallel workers, debate/critic, or event-driven.
- 4. Design shared state** - Use a structured task object with goal, plan, artifacts, results, errors, and status.
- 5. Define tool contracts** - Typed input/output, idempotency, timeouts, retry semantics, authorization.
- 6. Add planning and re-planning** - Create plan; execute; evaluate; revise only when evidence warrants it.
- 7. Add evaluation gates** - Check correctness, completeness, safety, policy, and business rules.
- 8. Add human-in-the-loop** - Require approval for irreversible, privileged, financial, or customer-impacting actions.
- 9. Add observability** - Distributed trace across agents, tool calls, costs, retries, and state transitions.
- 10. Add stop conditions** - Max steps, max cost, max retries, timeouts, confidence threshold, manual escalation.

Part XI - Detailed Agentic AI Examples

Example 1 - Multi-Agent Production Incident Response

Objective: investigate a production incident, restore service safely, and create a complete incident record.

Supervisor agent: Owns objective, assigns tasks, maintains incident state.

Observability agent: Queries metrics/logs/traces and summarizes anomalies.

Kubernetes/AWS diagnostics agent: Checks pods, deployments, IAM identity, DNS, networking, service health.

Change-analysis agent: Reviews recent PRs/deployments/config changes.

Communication agent: Drafts status updates for responders and management.

Evaluator: Checks whether evidence supports the proposed root cause before remediation.

Workflow: Execution: alert arrives -> supervisor creates hypotheses -> workers collect evidence in parallel -> evaluator ranks hypotheses -> human approves mitigation -> system verifies recovery -> communication agent publishes update -> postmortem draft is generated.

Why agentic: multiple roles coordinate against a shared state and re-plan based on evidence.

Example 2 - Autonomous Research and Report System

Objective: produce a sourced executive report on a complex topic.

Planner: Breaks the question into research subtopics.

Research agents: Search separate source categories in parallel.

Evidence agent: Deduplicates sources and extracts claims.

Critic: Looks for contradictions, unsupported conclusions, and missing perspectives.

Writer: Builds the final report from approved evidence.

Verifier: Checks citations and whether every major claim has support.

Workflow: Execution: objective -> plan -> parallel research -> evidence normalization -> critic feedback -> targeted follow-up research -> final synthesis -> verification.

Why agentic: progress is evaluated and the plan changes when evidence is incomplete.

Example 3 - Software Delivery Agentic System

Objective: implement a feature from issue to tested pull request.

Requirements agent: Clarifies acceptance criteria and identifies impacted modules.

Coding agent: Implements changes in a branch.

Test agent: Generates/runs unit and integration tests.

Security/review agent: Checks vulnerabilities, secrets, unsafe patterns, and code quality.

CI agent: Reads pipeline failures and sends targeted fixes back to coding agent.

Release assistant: Drafts PR summary, risk notes, rollback plan.

Workflow: Execution: issue -> plan -> code -> tests -> review -> CI feedback loop -> human review -> merge readiness.

Why agentic: specialized agents share artifacts, react to each other's outputs, and iterate until quality gates are satisfied.

Part XII - Agentic Architecture Patterns

Planner-Executor: Planner creates subtasks; executor performs them; evaluator checks progress. Good for long multi-step objectives.

Supervisor-Workers: Supervisor delegates to specialized agents. Good for incidents, research, and enterprise operations.

Sequential Pipeline: Each agent transforms the previous output. Easy to reason about but can propagate early errors.

Parallel Workers: Independent tasks run concurrently and merge later. Good for research, analysis, and validation.

Generator-Critic: One agent creates; another reviews and requests revision. Good for quality-sensitive outputs.

Event-Driven Agents: Agents react to queue/events. Good for operations, support, and asynchronous workflows.

Part XIII - Memory, Tools, Guardrails & Reliability

Memory

- **Working memory:** current task context and intermediate state.
- **Episodic memory:** summaries of past runs/interactions.
- **Semantic memory:** durable facts or knowledge extracted from past events.
- **External state store:** database/Redis/object store holding authoritative task state.

Tool design

- Use narrow tool functions with typed schemas.
- Make mutating operations idempotent where possible.
- Return machine-readable errors.
- Set timeouts and retry policies.
- Separate read-only and write-capable tools.
- Do not let the model construct arbitrary credentials or permissions.

Guardrails

- Authorization at the tool boundary.
- Human approval for destructive or privileged actions.
- Max steps, token budget, and cost budget.
- Input/output validation.
- Prompt-injection defenses for retrieved/untrusted content.
- Audit log for every external action.

Part XIV - Production Failure Modes: Detect and Fix

Failure	Signal / detection	Typical fix
Agent loop never stops	High step count, repeated tool calls, token/cost spike.	Add explicit stop conditions, max steps, state-based completion checks.
Tool retries duplicate actions	Duplicate tickets/orders/messages.	Idempotency keys, status lookup before retry, transactional semantics.
Wrong tool selected	Trace shows irrelevant action or permission error.	Improve tool descriptions, narrow tool set, routing classifier.
Stale memory	Agent uses old facts after source changes.	Timestamp memory, source-of-truth validation, TTL/retrieval refresh.
RAG returns unauthorized data	Cross-tenant or restricted document appears.	Enforce access control at retrieval layer; metadata ACL filters.
Planner makes bad decomposition	Tasks are redundant or impossible.	Plan validation, deterministic constraints, critic/evaluator.
Multi-agent deadlock	Agents wait on each other or bounce tasks.	Explicit ownership/state machine, timeouts, supervisor arbitration.
Prompt injection via document	Retrieved content instructs model to call dangerous tool.	Treat retrieved content as untrusted; policy filter; tool authorization.
Cost explosion	Many long model calls or large context.	Budgets, caching, summarization, cheaper model routing, max retries.
Latency explosion	Sequential agents/tools accumulate delay.	Parallelize safe subtasks, use smaller models, timeouts, cached retrieval.

Part XV - Evaluation & Observability

You cannot operate agents reliably if you only evaluate the final text. Track the full trajectory.

Suggested metrics

- **Task success rate:** did the objective actually complete?
- **Tool-call success rate:** percentage of calls that returned valid results.
- **Human correction rate:** how often people edit or override the result.
- **Escalation rate:** how often the agent appropriately hands off.
- **Steps per task:** detects loops and inefficient planning.
- **Latency:** end-to-end and per model/tool.
- **Token/cost per successful task:** more meaningful than raw token usage.
- **Retrieval precision/recall:** whether RAG returns the evidence needed.
- **Safety/policy violation rate:** tool misuse, access violations, unsupported actions.

Trace every run with: input -> plan -> model decisions -> tool inputs/outputs -> state changes -> evaluator results -> final outcome.

Part XVI - Interview Questions & Answer Guide

Q: What is the difference between an LLM and an AI agent?

Strong answer: An LLM predicts/generates language from context. An agent wraps a model with goals, state, tools, and a loop that can choose and execute actions.

Q: What is RAG and when would you use it?

Strong answer: RAG retrieves external knowledge and supplies it to the model. Use it when answers depend on private, current, or domain-specific information that should remain outside model weights.

Q: RAG vs fine-tuning?

Strong answer: RAG changes the context at inference time and is ideal for dynamic factual knowledge. Fine-tuning changes model behavior/style or task specialization. They can be combined.

Q: What is agentic AI?

Strong answer: A system pattern where one or more agents plan, act, observe, evaluate progress, and re-plan toward a broader objective.

Q: Single agent vs multi-agent?

Strong answer: Prefer a single agent when one workflow and tool set is sufficient. Add multiple agents only when role specialization, parallelism, isolation, or independent evaluation provides real value.

Q: How do you prevent an agent from looping forever?

Strong answer: Explicit completion criteria, max steps, time/cost budget, repeated-action detection, evaluator checks, and human escalation.

Q: How do you secure tool use?

Strong answer: Authenticate the workload, authorize at the tool boundary, use least privilege, schema validation, allowlists, human approval for high-risk actions, and audit every action.

Q: How do you handle prompt injection in RAG?

Strong answer: Treat retrieved data as untrusted, separate instructions from evidence, prevent content from granting permissions, restrict tools independently, and filter/validate actions.

Q: How do you evaluate an agent?

Strong answer: Measure task success, correctness, tool-call success, human corrections, steps, latency, cost, retrieval quality, and policy violations using realistic test scenarios.

Q: What is memory in an agent?

Strong answer: Stored state that persists beyond the immediate prompt. Distinguish task state, short-term conversation memory, and durable semantic/episodic memory.

Q: What is the main risk of multi-agent systems?

Strong answer: Complexity: coordination errors, duplicated work, deadlocks, cost/latency, and harder observability. Use them only when specialization or parallelism justifies it.

Q: How would you make agent actions idempotent?

Strong answer: Use operation IDs/idempotency keys, check current state before retrying, design create/update APIs safely, and persist action status.

Q: What does a planner do?

Strong answer: Transforms a broad objective into ordered or parallel subtasks, tracks dependencies, and revises the plan based on results.

Q: What does an evaluator/critic do?

Strong answer: Checks whether an intermediate or final result satisfies correctness, completeness, policy, or goal criteria and can trigger revision.

Q: How would you productionize an agent?

Strong answer: Typed tools, auth, RAG ACLs, durable state, guardrails, retries/timeouts, traceability, offline/online evaluations, budgets, alerts, rollback/kill switch, and human approval for risky actions.

Part XVII - System Design Interview Prompts

Design a support agent

Cover ticket ingestion, classification, RAG over runbooks, tool calls to ticketing systems, confidence thresholds, escalation, audit logs, and evaluation.

Design an SRE incident agent

Cover alert ingestion, read-only telemetry tools, deployment/change history, hypothesis ranking, human-approved mitigation, incident state, communications, and postmortem.

Design a research agent

Cover query decomposition, parallel research, source ranking, evidence store, citation verification, contradiction handling, writer + critic loop.

Design an agent that modifies production

Lead with risk controls: scoped IAM, approvals, dry-run, policy engine, idempotency, change windows, rollback, traceability, kill switch.

Design an enterprise RAG agent

Cover ingestion, parsing/chunking, embeddings, hybrid retrieval, metadata ACLs, reranking, citations, freshness, multi-tenancy, observability.

Part XVIII - Quick Interview Cheat Sheet

Term	Remember
LLM	Generates from context; no built-in guarantee of fresh/private knowledge.
RAG	Retrieve knowledge first, then generate grounded answer.
Agent	LLM + goal + tools + state + decision/action loop.
Agentic AI	Planning + acting + observing + evaluating + re-planning, possibly across multiple agents.
Memory	Persisted state; use deliberately, not automatically.
Tool	Typed capability the agent can invoke; secure independently from the model.
Orchestrator	Coordinates tasks/agents and shared state.
Guardrail	Constraint that prevents invalid, unsafe, or unauthorized behavior.
Evaluator	Checks whether work meets goal/quality/policy criteria.
Observability	Full trace of model/tool/state transitions plus cost/latency/outcomes.

Best interview framing: start with the simplest architecture that can solve the problem. Add RAG, memory, planning, or multiple agents only when the requirement justifies the extra complexity.